**Lab Assignment: Smart Payment System**

You are tasked with designing a **Smart Payment System** that supports different payment methods (Credit Card, PayPal, and Cryptocurrency). A user can request a payment processor by name, and your system must:

1. Create the processor dynamically using a **Factory Method**.
2. Process the payment polymorphically via an **interface**.
3. Handle invalid inputs with proper **exception handling**.

**Class Specifications**

**1) PaymentProcessor (Interface)**

- Method: `void processPayment(double amount) throws PaymentException`

**2) Implementations**

- **CreditCardProcessor**
  - Validates that amount $\leq 5000$ (otherwise throws `PaymentException`).
  - Prints: `"Processing credit card payment of <amount>"`.
- **PayPalProcessor**
  - Validates that amount $\geq 10$ (otherwise throws `PaymentException`).
  - Prints: `"Processing PayPal payment of <amount>"`.
- **CryptoProcessor**
  - Validates that amount is divisible by 5 (otherwise throws `PaymentException`).
  - Prints: `"Processing crypto payment of <amount>"`.

**3) PaymentException (Custom Exception)**

- Extends `Exception`.
- Constructor takes a message string.
- Used for invalid payment amounts.

**4) PaymentFactory (Factory Method)**

- Private constructor (to prevent direct instantiation).
- Public static method:

```
public static PaymentProcessor createProcessor(String type)
        throws IllegalArgumentException
```

- Returns the correct processor (`CreditCardProcessor`, `PayPalProcessor`, `CryptoProcessor`).
- Throws `IllegalArgumentException` if the type is unknown.

### Program Requirements (Main Program)

1. Ask the user for:
   - Payment type (`credit`, `paypal`, `crypto`).
   - Payment amount.
2. Use `PaymentFactory.createProcessor(type)` to obtain the processor.
3. Call `processPayment(amount)`.
4. Wrap logic in try-catch blocks to handle:
   - `PaymentException` (invalid amount).
   - `IllegalArgumentException` (unknown type).
5. Display meaningful error messages when exceptions occur.

### Example Console Output

#### Run 1 (valid)

```
Enter payment type: credit
Enter amount: 2000
Processing credit card payment of 2000.0
```

#### Run 2 (invalid amount)

```
Enter payment type: paypal
Enter amount: 5
Error: PayPal requires a minimum payment of 10
```

#### Run 3 (unknown type)

```
Enter payment type: bank
Enter amount: 100
Error: Unknown payment type: bank
```

### Submission Instructions

- **Platform:** Google Classroom
- **Submit:**
  - A .txt file answering and discussing Reflection Questions.
  - Source files:
    1. `PaymentProcessor.java`,
    2. `CreditCardProcessor.java`,
    3. `PayPalProcessor.java`,
    4. `CryptoProcessor.java`,

5. `PaymentException.java`,
6. `PaymentFactory.java`,
7. `SmartPaymentSystem.java` (main).

– Three screenshots showing:

1. A successful transaction.
2. An invalid payment amount error.
3. An unknown payment type error.

**Reflection Questions**

1. Why is it better to define `PaymentProcessor` as an **interface** instead of a base class?
2. What advantages does the **factory method pattern** bring in terms of extensibility?
3. How does exception handling improve program **robustness and user experience**?